

ESP32-Based Wireless Embedded Control System for a Dual-Motor Robotic Vehicle

By: Jhostin Sanchez

Distributed Embedded Systems Project Using Dual ESP32 Microcontrollers

May 2026



Abstract

This project presents the design and implementation of an ESP32-based wireless embedded control system for a dual-motor robotic vehicle. The system utilizes two independent microcontrollers: an ESP32 mounted on the vehicle for real-time hardware control and an ESP32-S3 display module functioning as a wireless remote-control interface. Communication between both systems is established through Wi-Fi using the UDP protocol to achieve low-latency real-time control performance.

The embedded control architecture integrates motor control, wireless communication, peripheral management, and power stabilization into a single distributed system. An L293D motor driver is used to control the rear drive motor and steering motor, while additional GPIO-controlled peripherals include an LED lighting system and buzzer module. PWM-based motor control enables variable speed operation and smoother driving response. To improve system reliability, separate motor and logic power rails were implemented alongside capacitor-based voltage stabilization techniques to reduce electrical noise and prevent controller instability during motor operation.

The project demonstrates the integration of embedded systems engineering, wireless communication, power management, and real-time control logic within a compact robotic platform. The final system achieves stable wireless operation, responsive vehicle control, and modular expandability for future improvements including PCB integration, advanced telemetry, and enhanced motor driver architectures.

Introduction

Embedded systems and wireless control technologies play a major role in modern robotics, automation systems, autonomous platforms, and intelligent electronic devices. Real-time communication between distributed controllers is commonly used in applications such as robotic vehicles, industrial automation, smart transportation systems, and remote-operated machines. As embedded hardware becomes more powerful and compact, microcontrollers such as the ESP32 enable the development of advanced wireless systems capable of integrating communication, sensing, processing, and hardware control into a single platform.

This project focuses on the development of a wireless embedded control system for a dual-motor robotic vehicle using two ESP32-based microcontrollers. The system was designed to demonstrate real-time wireless vehicle control, distributed embedded processing, PWM motor speed regulation, and power stabilization techniques within a compact robotic platform. The architecture separates the system into two independent subsystems: a handheld ESP32-S3 remote controller responsible for user interaction and wireless command transmission, and an ESP32-based vehicle controller responsible for motor control and hardware execution.

Communication between both systems is achieved through Wi-Fi using the UDP protocol to reduce communication latency and improve responsiveness during operation. The vehicle utilizes an L293D motor driver for bidirectional motor control and integrates additional peripherals including an LED lighting system and buzzer module. Special attention was given to power system architecture by separating logic and motor power supplies to improve electrical stability and prevent controller resets caused by motor current fluctuations.

The objective of this project is not only to create a functional wireless robotic vehicle, but also to demonstrate important embedded systems engineering concepts including hardware-software integration, distributed wireless communication, PWM-based motor control, power management, and modular system design. The final platform provides a foundation for future expansion into more advanced robotic systems, custom PCB integration, telemetry systems, autonomous navigation, and higher-efficiency motor driver architectures.

Objectives

The primary objectives of this project were:

- Design and develop a wireless embedded control system for a dual-motor robotic vehicle using ESP32 microcontrollers.
- Implement real-time wireless communication between an ESP32-S3 remote controller and an ESP32 vehicle controller using Wi-Fi and UDP protocol architecture.
- Develop independent drive and steering motor control using the L293D motor driver with bidirectional motor operation.
- Integrate PWM-based speed control to achieve variable motor speed regulation and smoother vehicle movement.
- Implement auxiliary peripheral control including an LED lighting system and buzzer module through GPIO-based output control.
- Design a stable power distribution architecture by separating motor power and logic power supplies to improve system reliability.
- Apply capacitor-based voltage stabilization techniques to reduce electrical noise and prevent ESP32 instability during motor operation.
- Create a modular system architecture that supports future expansion including PCB integration, telemetry systems, autonomous functions, and improved motor driver technologies.
- Demonstrate practical embedded systems engineering concepts including hardware-software integration, wireless communication, real-time processing, and power management within a compact robotic platform.

Components & Materials

Component	Quantity	Function
ESP32 Development Board	1	Main vehicle controller responsible for wireless communication, GPIO control, PWM generation, and hardware execution
ESP32-S3 Display Module	1	Wireless remote-control interface used for joystick input, touchscreen interaction, and UDP command transmission
L293D Motor Driver IC	1	Dual H-bridge motor driver used for bidirectional drive and steering motor control
DC Drive Motor	1	Provides forward and reverse propulsion for the robotic vehicle
DC Steering Motor	1	Controls left and right steering movement
LED	1	Visual indicator and lighting system
220Ω Resistor	1	Current-limiting resistor for LED protection
Active Buzzer Module	1	Audio output device used for horn and sound indication
100μF Capacitor	1	Voltage stabilization capacitor used to reduce power fluctuations during motor operation
0.1μF Ceramic Capacitor	1	High-frequency noise filtering capacitor for improved power stability
Breadboard Power Module	1	Supplies regulated 5V logic power to the ESP32 and control circuitry
AA Batteries	3	Dedicated motor power supply for the L293D motor voltage rail
Breadboard	1	Prototyping platform used for temporary circuit assembly
Jumper Wires	Múltiple	Electrical interconnections between components
USB-C Cable	1	Power and programming connection for the ESP32-S3 display module
Wi-Fi Communication Link	1	Wireless communication interface used for UDP-based command transmission

System Overview

The ESP32-based wireless embedded control system is a distributed robotic vehicle platform designed to achieve real-time remote control through wireless communication. The system is divided into two primary subsystems: a handheld ESP32-S3 remote controller and an ESP32-based onboard vehicle controller. These two microcontrollers operate independently but remain synchronized through Wi-Fi-based UDP communication, enabling low-latency command transmission and real-time system response.

The ESP32-S3 module functions as the user interface and command generation unit. It reads analog joystick inputs and digital button states, processes them into structured movement and peripheral control commands, and transmits these commands wirelessly over a Wi-Fi network using the UDP protocol. This allows continuous streaming of control data with minimal communication delay.

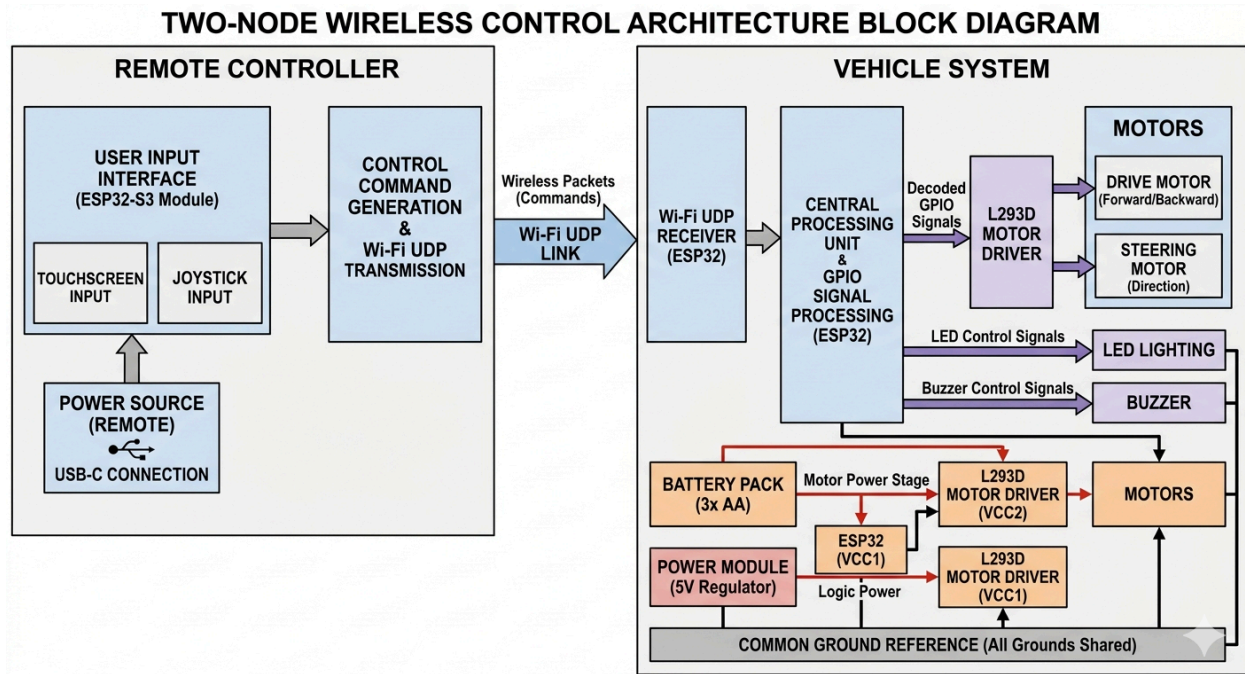
The ESP32 mounted on the robotic vehicle acts as the main execution controller. It receives incoming UDP packets, decodes the transmitted commands, and maps them to hardware-level actions. These actions include motor control via an L293D motor driver, LED state control, and buzzer activation. Pulse-width modulation (PWM) is used to regulate motor speed, enabling smooth acceleration and directional control for both drive and steering mechanisms.

The power system is divided into separate logic and motor supply domains to ensure electrical stability. The ESP32 and control logic are powered through a regulated 5V supply, while the motors are powered independently using a separate battery source connected to the L293D motor driver. Capacitors are used across the power rails to reduce voltage fluctuations and minimize noise generated by motor switching.

Overall, the system demonstrates a modular embedded architecture that integrates wireless communication, real-time processing, motor control, and power management. The separation of input processing, communication, and hardware execution enables stable and responsive robotic vehicle operation suitable for future expansion into PCB-based and more advanced autonomous systems.

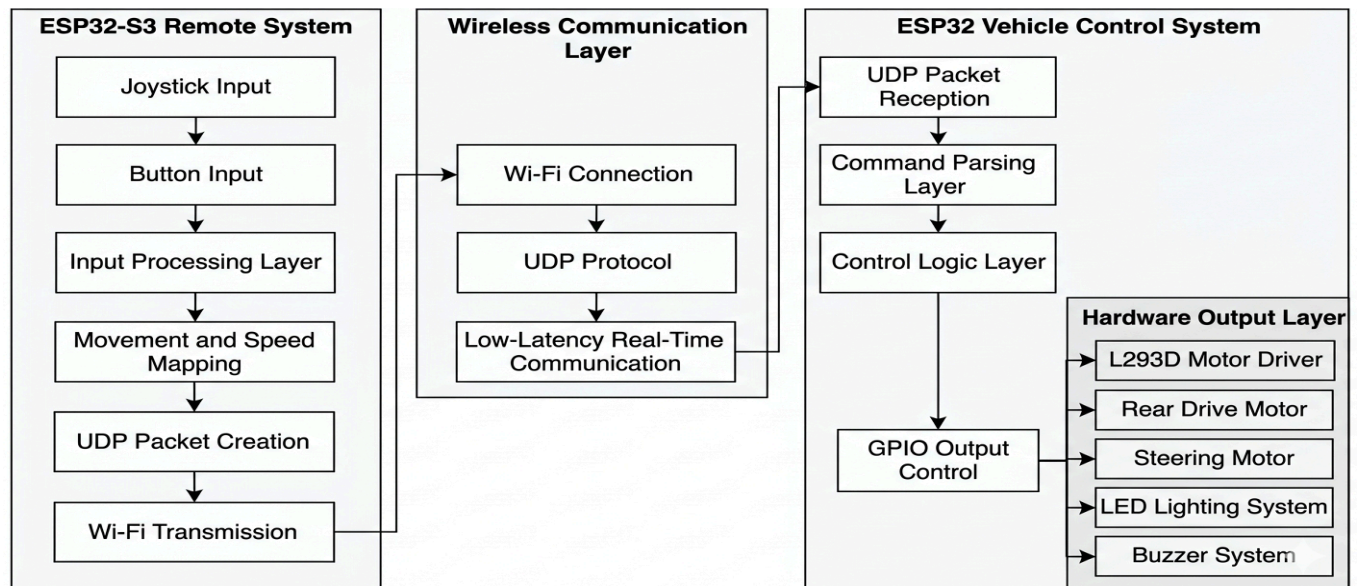
System Architecture

1. Hardware Block Diagram



Hardware architecture of the ESP32-based robotic system showing physical component interconnections.

2. Software Block Diagram



Software architecture showing data flow between ESP32-S3 remote controller and ESP32 vehicle controller via UDP communication.

Circuit Design

The circuit is designed to integrate the ESP32 microcontroller with motor control, wireless communication, and peripheral output systems, enabling real-time wireless control of a dual-motor robotic vehicle. The architecture separates signal processing, power distribution, and output control to ensure stable and reliable operation.

7.1 Circuit Design Compilation

Motor Control Subsystem

- ESP32 GPIO → L293D IN1, IN2 (Drive Motor Direction)
- ESP32 GPIO → L293D IN3, IN4 (Steering Motor Direction)
- ESP32 PWM GPIO → L293D ENA (Speed Control)
- ESP32 PWM GPIO → L293D ENB (Speed Control)
- L293D OUT1/OUT2 → Drive Motor
- L293D OUT3/OUT4 → Steering Motor

Power Distribution Subsystem

- 5V Power Rail → ESP32 VIN
- 5V Power Rail → L293D VCC1 (Logic Supply)
- 3× AA Battery Pack → L293D VCC2 (Motor Supply)
- Common Ground → ESP32, L293D, Power Module, Battery Pack
- 100μF Capacitor → 5V Rail Stabilization
- 0.1μF Capacitor → Noise Filtering

Peripheral Output Subsystem

- ESP32 GPIO → LED → 220Ω Resistor → GND
- ESP32 GPIO → Active Buzzer → GND

Wireless Control Interface

- ESP32-S3 Remote → Wi-Fi Network (UDP Transmission)
- ESP32 Vehicle → Wi-Fi Receiver (UDP Reception)
- Command Packet → Movement, Speed, Peripheral States

7.2 Pin Configuration (ESP32)

Function	GPIO Pin
Drive Motor IN1	GPIO 26

Drive Motor IN2	GPIO 25
Drive Motor PWM (ENA)	GPIO 5
Steering Motor IN3	GPIO 27
Steering Motor IN4	GPIO 14
Steering PWM (ENB)	GPIO 18
Buzzer	GPIO 23
LED	GPIO 22

7.3 Circuit Stability and Design Considerations

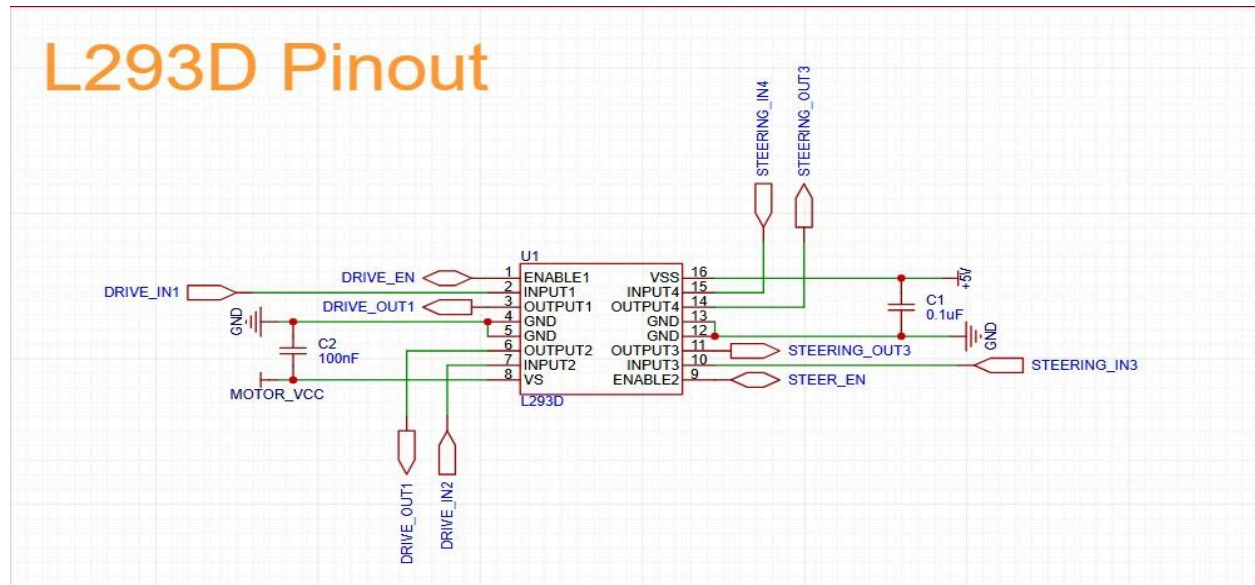
- Separation of motor and logic power domains
- Shared ground reference across all subsystems
- Capacitor-based voltage stabilization for motor-induced fluctuations
- PWM-based speed regulation for controlled motor response
- Isolation of wireless communication stability from motor load variations

7.4 System Integration Summary

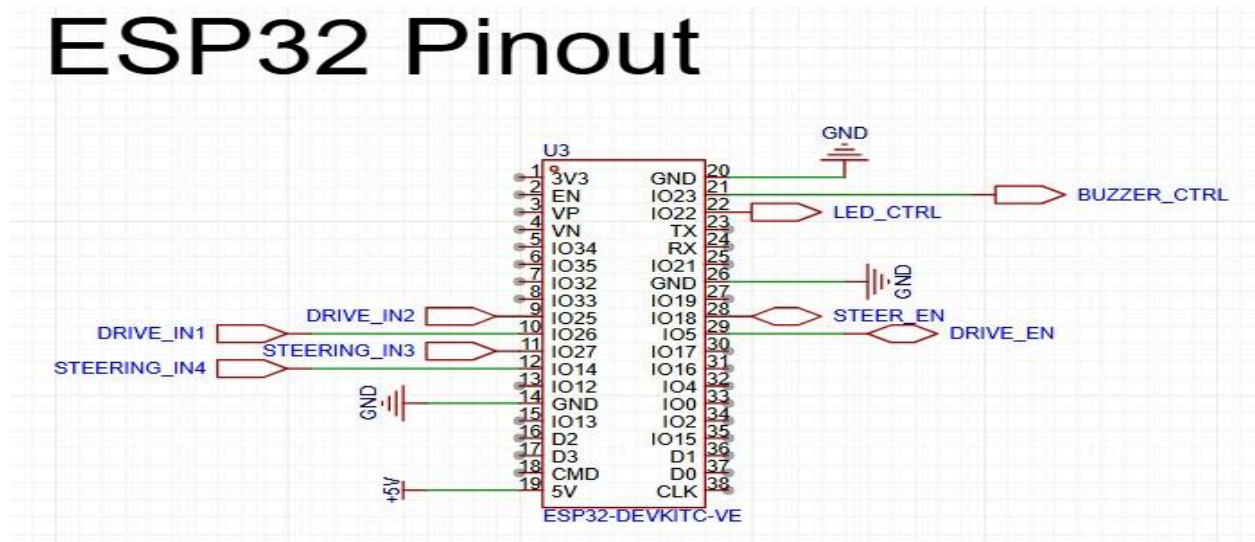
The circuit architecture combines embedded processing, motor actuation, and wireless communication into a unified system. The ESP32 manages real-time control signals while the L293D interfaces high-current motor loads. Peripheral components and power regulation ensure stable operation under dynamic load conditions.

7.5 Circuit Schematics

L293D Schematic

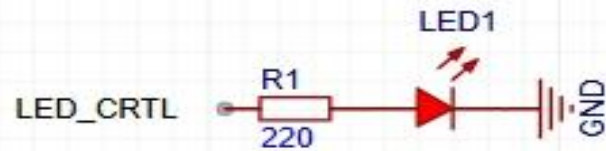


ESP32 Schematic



LED Indicator Schematic

LED Indicator



Motors Schematic

Rear Drive Motor

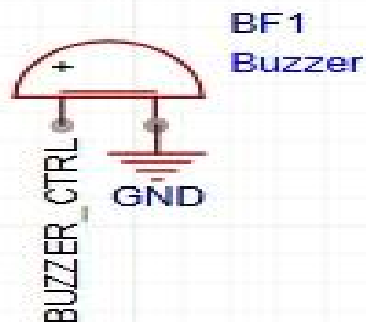


Steering Motor



Buzzer Schematic

BUZZER SOUND



Circuit Calculations and Electrical Analysis

8.1 LED Electrical Model

$$I = \frac{V_S - V_F}{R}$$

$$P = VI$$

$$I = \frac{5V - 2V}{220\Omega} = 13.64mA$$

$$P = (0.01364A)(3V) = 40.9mW$$

8.2 PWM Motor Drive Model

$$D = \frac{T_{ON}}{T_{Total}} \times 100\%$$

$$V_{avg} = D \cdot V_{supply}$$

$$V_{supply} = 4.5V$$

$$V_{avg} = 0.2(4.5) = 0.9V, 0.5(4.5) = 2.25V, 1.0(4.5) = 4.5V$$

8.3 DC Motor Electrical Load Characteristics

$$P = VI$$

$$P_{nominal} = (4.5V)(0.15A) = 0.675W$$

$$P_{stall} = (4.5V)(0.6A) = 2.7W$$

$$I_{stall} \approx 4I_{nominal}$$

8.4 System Power Budget and Distribution

$$I_{total} = I_{logic} + I_{motor}$$

$$I_{logic} \approx 0.18A$$

$$I_{motor(max)} \approx 0.6A$$

$$I_{total(max)} = 0.78A$$

$$P_{total} = V \cdot I = 5V \cdot 0.78A = 3.9W$$

8.5 Energy Decoupling Between Logic and Motor Domains

$$P = VI$$

The system operates with two distinct power domains:

- Logic domain (ESP32 + peripherals): 5V regulated supply
- Motor domain (L293D + DC motors): 4.5V battery supply

Total current demand during simultaneous operation:

$$I_{total} = I_{ESP32} + I_{motors} + I_{peripherals}$$

Using realistic embedded load estimates:

$$I_{ESP32} \approx 0.18A, I_{motors} \approx 0.6A, I_{LED+buzzer} \approx 0.02A, I_{total} \approx 0.8A$$

System constraint:

- motor current spikes must not reflect into ESP32 supply rail
- voltage stability is critical for Wi-Fi operation

8.6 Capacitor Charge and Voltage Stabilization

Capacitor behavior:

$$Q = CV$$

For 100 μ F capacitor at 5V:

$$Q = (100 \times 10^{-6} F)(5V) = 5 \times 10^{-4} C$$

Current response relation:

$$I = C \frac{dV}{dt}$$

Implication:

- high motor switching $\rightarrow$ rapid current demand
- capacitor supplies transient current locally
- reduces voltage dip duration at ESP32 rail

8.7 DC Motor Electrical Behavior (Real Model)

Motor power consumption range:

$$P = VI$$

Nominal operating condition:

$$P_{nominal} = (4.5V)(0.15A) = 0.675W$$

Startup / stall condition:

$$P_{stall} = (4.5V)(0.6A) = 2.7W$$

Motor current scaling:

$$I_{stall} \approx 4 \times I_{nominal}$$

8.8 LED Electrical Safety Constraint

$$I = \frac{V_S - V_F}{R}$$

$$I = \frac{5V - 2V}{220\Omega} = 13.64mA$$

Power dissipation:

$$P = VI = (0.01364A)(3V) = 40.9mW$$

Constraint:

- safe for ESP32 GPIO output
- prevents LED thermal overstress

8.9 L293D Voltage Drop and Efficiency Loss

The L293D motor driver operates using bipolar transistor output stages, resulting in internal voltage losses during motor operation.

Internal output voltage drop:

$$V_{motor} = V_{supply} - V_{drop}$$

Typical L293D voltage drop per channel:

- $\approx 1.2V - 1.8V$ under load

For a 4.5V motor supply:

$$V_{motor} = 4.5V - 1.4V = 3.1V$$

Effective motor voltage:

- $\approx 3.0V - 3.3V$ during normal operation

Power dissipated internally by the L293D:

$$P_{loss} = V_{drop} \cdot I$$

Nominal operating condition:

$$P_{loss} = (1.4V)(0.15A) = 0.21W$$

High-load / startup condition:

$$P_{loss} = (1.4V)(0.6A) = 0.84W$$

Driver efficiency approximation:

$$\eta = \frac{P_{motor}}{P_{input}} \times 100\%$$

Approximate efficiency:

$$\eta = \frac{3.1V}{4.5V} \times 100\% \approx 68.9\%$$

Electrical implications:

- reduced motor torque under load
- reduced maximum motor speed
- thermal energy dissipation within driver IC
- increased battery power consumption
- reduced overall system efficiency

Engineering limitation:

- bipolar transistor H-bridge architecture introduces significant conduction losses compared to MOSFET-based drivers

Future improvement path:

- replacement with TB6612FNG or MOSFET motor driver architecture for improved efficiency and reduced thermal loss

8.10 Battery Runtime and Discharge Analysis

The motor subsystem is powered using a $3 \times$ AA battery configuration supplying approximately 4.5V under nominal conditions.

Battery pack voltage:

$$V_{battery} = 1.5V + 1.5V + 1.5V = 4.5V$$

Typical alkaline AA battery capacity:

- $\approx 2000\text{mAh}$ under moderate load conditions

Series battery configuration characteristics:

- voltage increases
- capacity remains constant

Effective battery pack capacity:

$$C_{total} \approx 2000\text{mAh} = 2\text{Ah}$$

Battery runtime estimation:

$$t = \frac{C}{I}$$

Nominal operating current:

$$t = \frac{2\text{Ah}}{0.15\text{A}} \approx 13.3\text{h}$$

High-load operating condition:

$$t = \frac{2\text{Ah}}{0.6\text{A}} \approx 3.3\text{h}$$

Practical runtime reduction factors:

- motor startup surge current
- L293D efficiency losses
- internal battery resistance
- voltage sag under dynamic load
- intermittent steering load spikes

Battery voltage behavior under load:

$$V_{load} = V_{nominal} - I(R_{Internal})$$

electrical implications:

- increasing current demand produces additional voltage drop
- lower battery voltage reduces available motor torque
- unstable voltage conditions increase ESP32 communication instability risk
- prolonged operation decreases overall motor responsiveness

System-level observation:

- battery-powered motor isolation improves wireless communication stability by preventing high-current transients from propagating into the ESP32 logic rail

Engineering limitation:

- alkaline AA batteries exhibit reduced effective capacity under high-current pulse loads

Future improvement path:

- rechargeable lithium-ion battery system
- regulated buck converter architecture
- battery management and protection circuitry
- real-time voltage telemetry and monitoring

8.11 Thermal and System-Level Engineering Constraints

The embedded control system operates under dynamic electrical and thermal loading conditions generated by simultaneous motor actuation, wireless communication, and peripheral control.

Total estimated system power:

$$P_{system} = VI$$

Peak operating condition:

$$P_{system} = (5V)(0.78A) \approx 3.9W$$

L293D thermal dissipation:

$$P_{thermal} = V_{drop} \cdot I$$

Peak thermal condition:

$$P_{thermal} = (1.4V)(0.6A) = 0.84W$$

Thermal implications:

- internal transistor heating within L293D package
- reduced efficiency during prolonged operation
- increased thermal stress during stall conditions
- elevated power dissipation during simultaneous motor activation

Electrical system constraints:

- breadboard interconnect resistance
- jumper wire contact instability
- transient switching noise
- voltage fluctuation during motor acceleration
- limited current handling capability of temporary prototyping connections

Signal integrity constraints:

$$\Delta V \propto I_{switching}$$

System behavior observations:

- increased motor load produces greater voltage instability
- unstable voltage conditions increase wireless communication interruption probability
- capacitor filtering improves transient suppression during rapid load transitions
- PWM modulation reduces abrupt current demand changes compared to direct switching

Mechanical constraints:

- increased breadboard mass reduces vehicle mobility efficiency
- exposed wiring reduces structural reliability
- temporary chassis modifications limit long-term durability
- steering subsystem experiences elevated mechanical resistance during directional transitions

Embedded system limitations:

- absence of current sensing circuitry
- absence of battery protection circuitry
- absence of thermal monitoring
- limited motor driver efficiency
- limited scalability using breadboard architecture

Engineering design justification:

- separation of logic and motor power domains improves embedded system reliability
- capacitor-assisted voltage stabilization improves wireless communication consistency
- PWM-based control architecture enables smoother motor response and reduced instantaneous current demand

Future engineering improvements:

- custom PCB implementation
- MOSFET-based motor driver architecture
- integrated battery management system
- onboard current monitoring
- thermal protection circuitry
- enclosed mechanical chassis
- optimized power distribution network
- higher-efficiency motor control topology

Certain electrical values were estimated using nominal component specifications and measured operating behavior due to variability in motor load conditions during dynamic operation.

System Operation

The system operates as a distributed real-time embedded control architecture composed of a wireless remote controller (ESP32-S3) and an onboard vehicle control unit (ESP32). Upon system startup, the ESP32 initializes as a Wi-Fi access point and begins listening for incoming UDP packets on a predefined communication port. Simultaneously, the ESP32-S3 connects to the access point and begins continuously sampling joystick and button inputs from the user interface.

During operation, joystick displacement values are continuously converted into directional and speed commands. These values are encoded into lightweight UDP packets and transmitted over a low-latency Wi-Fi link. The communication layer operates without handshake confirmation, enabling near real-time streaming of control data.

On the vehicle side, the ESP32 continuously listens for incoming packets and parses the received data into three main control categories: motion direction, motor speed, and peripheral states. Based on decoded values, the ESP32 updates GPIO outputs in real time.

Motor control signals are routed through the L293D motor driver, where directional logic signals determine forward, reverse, left, or right movement, while PWM signals control speed modulation of both the drive and steering subsystems. The LED indicator system responds directly to GPIO state changes, providing visual feedback for system status, while the buzzer is activated based on user input events transmitted from the remote controller.

The system operates in a continuous closed-loop interaction where input sampling, wireless transmission, decoding, and actuator response occur repeatedly in real time. This creates a responsive embedded control loop where user joystick movement directly corresponds to vehicle motion with minimal perceived latency.

To ensure stability during operation, decoupling capacitors smooth transient voltage drops caused by motor startup current, preventing resets in the ESP32 and maintaining consistent wireless communication performance. The separation of motor and logic power domains ensures that high-current motor activity does not interfere with communication reliability.

Overall, the system behaves as a real-time wireless embedded control platform where input acquisition, wireless communication, and actuator response are synchronized through continuous UDP-based streaming and GPIO-level motor control.

Testing and Results

The system was evaluated through a series of functional and performance tests focused on wireless communication reliability, motor response accuracy, and overall embedded system stability under real operating conditions.

10.1 Wireless Communication Test (ESP32-S3 → ESP32)

The first test validated UDP-based communication between the remote controller and the vehicle. The ESP32-S3 transmitted joystick and button data at a consistent refresh rate while the ESP32 received and parsed incoming packets in real time.

Observed behavior:

- Stable connection within Wi-Fi access point range
- Low-latency transmission suitable for real-time control
- No required handshake delays due to UDP protocol usage
- Occasional packet loss under extreme interference, but system recovered immediately on next update cycle

Overall result:

- Communication system performed reliably for continuous control input streaming

10.2 Motor Response Test (Drive + Steering System)

The motor subsystem was tested by applying incremental joystick inputs and observing directional and speed response.

Observed behavior:

- Forward and reverse motion responded proportionally to joystick displacement
- Steering motor reacted immediately to directional input changes
- PWM-based speed control produced smooth acceleration and deceleration
- No abrupt switching when properly stabilized by capacitor filtering

Limitations observed:

- Slight delay under peak load conditions due to L293D voltage drop
- Reduced torque during simultaneous steering and driving commands

Overall result:

- Motor control system functioned as intended with acceptable real-time responsiveness

10.3 Power Stability and Reset Test

System stability was tested under full load conditions including simultaneous motor operation, Wi-Fi transmission, LED activation, and buzzer usage.

Observed behavior:

- Initial instability when motors shared unstable power source
- Voltage dips caused occasional ESP32 resets during startup conditions
- After separation of logic and motor power rails, system stability significantly improved
- Capacitors reduced transient dips during motor activation

Overall result:

- Power architecture modification successfully resolved major reset and Wi-Fi dropout issues

10.4 Latency and Responsiveness Test

Responsiveness was evaluated by rapidly changing joystick input and observing vehicle reaction time.

Observed behavior:

- Near real-time response between input and motion
- UDP communication contributed to minimal transmission delay
- Slight lag noticeable only during high motor load transitions

Estimated performance:

- Perceived latency remained low enough for interactive remote control operation

10.5 System Reliability Test

The system was run continuously over extended periods to evaluate stability.

Observed behavior:

- Consistent operation over multiple minutes without failure after stabilization improvements
- No persistent disconnection after power architecture correction
- Stable GPIO output behavior across all subsystems

Overall Testing Conclusion

The system successfully demonstrated:

- Functional real-time wireless control
- Stable ESP32-to-ESP32 communication using UDP
- Reliable motor actuation through L293D driver
- Improved power stability after circuit optimization

Remaining limitations include:

- Efficiency loss in motor driver stage
- Breadboard-based wiring instability
- Lack of formal telemetry feedback system

Challenges and Limitations

During the development of the wireless RC car system, several technical and physical constraints were encountered that directly influenced system design decisions, performance, and final implementation. These challenges were primarily related to power distribution, wireless stability, motor driving efficiency, and prototyping limitations.

11.1 Power Instability and Voltage Drops

One of the primary challenges was unstable power delivery during motor operation. When both motors activated simultaneously, current spikes caused voltage dips that affected the ESP32 stability, resulting in occasional resets and Wi-Fi disconnections. This issue was traced to shared power conditions between high-current motor loads and sensitive logic circuitry.

The problem was mitigated by separating power domains, using a dedicated 3× AA battery pack for motor supply while maintaining a regulated 5V logic rail for the ESP32 and peripheral components. Additionally, decoupling capacitors were introduced to reduce transient voltage fluctuations during motor startup.

11.2 Wireless Communication Instability (Initial Phase)

Early testing revealed intermittent Wi-Fi instability, especially during high motor load conditions. These disruptions were caused by electrical noise generated by motor switching and insufficient power isolation.

After implementing improved power separation and capacitor-based filtering, system stability significantly improved, allowing consistent UDP-based communication between the ESP32-S3 remote controller and the ESP32 vehicle unit.

11.3 Motor Driver Efficiency Limitations

The L293D motor driver introduced noticeable voltage drops due to its internal bipolar transistor structure. This reduced the effective voltage delivered to the motors, resulting in lower torque and reduced overall efficiency.

Under higher load conditions, the driver also exhibited thermal losses, further limiting performance. This constraint highlighted the need for a more efficient motor driver solution in future revisions.

11.4 Prototyping and Wiring Constraints

The system was implemented on a breadboard-based architecture, which introduced physical limitations such as:

- Loose jumper wire connections
- Increased electrical resistance at contact points
- Mechanical instability due to vibration during motor movement
- Increased circuit complexity and reduced reliability under motion

These limitations affected long-term durability and highlighted the necessity of a PCB-based design for future versions.

11.5 Mechanical System Constraints

The physical assembly of the vehicle introduced additional limitations in structural design and motor load distribution. The steering mechanism required continuous force application, increasing current demand compared to standard rotational motor systems.

The absence of a fully enclosed chassis also exposed wiring and components, limiting mechanical robustness and scalability.

11.6 System Complexity and Integration Challenges

Integrating multiple subsystems, including wireless communication, motor control, and peripheral outputs required careful synchronization between hardware and software components. Ensuring that UDP communication, PWM motor control, and GPIO outputs operated concurrently without interference was a key design challenge.

Overall Summary

Despite these challenges, each limitation contributed directly to system improvement and deeper understanding of embedded system design principles. The final implementation successfully overcame major instability issues through power domain separation, improved filtering, and optimized communication structure, resulting in a stable real-time wireless control system.

Future Improvements

Although the current wireless RC car system is fully functional and demonstrates real-time embedded control using dual ESP32 communication, several improvements can be implemented to enhance performance, efficiency, scalability, and long-term reliability.

12.1 PCB-Based System Design

The current breadboard implementation introduces wiring instability and increased electrical resistance. A custom-designed PCB would significantly improve:

- signal integrity
- mechanical durability
- power distribution efficiency
- overall system compactness

A PCB layout would also reduce noise caused by long jumper wires and improve reliability during motor switching events.

12.2 Motor Driver Upgrade (L293D Replacement)

The L293D motor driver introduces significant voltage drop and thermal inefficiency due to its bipolar transistor architecture.

A recommended upgrade would be:

- TB6612FNG or equivalent MOSFET-based H-bridge driver

Expected improvements:

- higher efficiency
- reduced heat generation
- improved motor torque output
- lower voltage loss under load

12.3 Battery System Optimization

The current AA battery system provides simplicity but limited performance under high current demand.

Future improvements include:

- Li-ion or LiPo battery pack integration
- voltage regulation using buck/boost converters
- battery management system (BMS) for protection and monitoring
- real-time voltage feedback to ESP32

12.4 Sensor Integration and Telemetry

The system currently operates as an open-loop control platform. Adding sensors would enable feedback-based control.

Possible additions:

- wheel speed sensors (encoder-based feedback)
- battery voltage monitoring
- current sensing module
- optional IMU for motion tracking

This would enable telemetry and more advanced control algorithms.

12.5 Software Enhancements

The current UDP-based control system is functional but can be improved for robustness and intelligence.

Future software upgrades:

- signal loss failsafe (auto-stop on disconnect)
- acceleration smoothing curves for more realistic motion
- packet acknowledgment system for reliability
- adaptive speed control based on load conditions

12.6 Mechanical Design Improvements

The physical structure of the vehicle can be optimized for performance and durability.

Future enhancements:

- fully enclosed chassis design
- optimized weight distribution
- improved steering mechanism efficiency
- vibration-resistant mounting for electronics

12.7 System Scalability

The current architecture can be expanded into a more advanced embedded platform.

Potential expansions:

- multi-vehicle control system
- camera-based FPV integration
- autonomous driving mode
- sensor fusion for obstacle avoidance

Overall Summary

The current system provides a strong foundation for real-time wireless embedded control. Future improvements primarily focus on transitioning from a prototype-level implementation to a fully optimized embedded engineering platform with improved efficiency, stability, and scalability.

Conclusion

The ESP32-Based Wireless Embedded Control System demonstrates the successful design and implementation of a real-time embedded control platform built around dual ESP32 microcontrollers communicating over a low-latency UDP-based Wi-Fi network. The project integrates multiple engineering domains including embedded systems programming, digital communication, power electronics, and electromechanical actuation into a single cohesive system.

Throughout development, the system evolved from a basic prototype into a stable multi-subsystem architecture. The ESP32-S3 remote controller effectively captures real-time user input through joystick and button interfaces, converting these signals into structured wireless commands. These commands are transmitted over Wi-Fi and interpreted by the onboard ESP32, which manages motor control through an L293D driver while simultaneously handling peripheral outputs such as LED indicators and a buzzer.

A key aspect of the system was resolving instability issues caused by power noise and motor current spikes. Initial testing revealed ESP32 resets and Wi-Fi interruptions during motor activation. These issues were mitigated through separation of power domains, implementation of decoupling capacitors, and improved distribution of load between motor and logic circuits. This directly improved system reliability and communication stability.

From an electrical engineering perspective, the project highlights important concepts such as voltage drop across motor drivers, PWM-based speed control, battery discharge behavior under load, and transient response effects in mixed-signal embedded systems. The analysis and calculations performed in earlier sections reinforce the physical limitations and real-world constraints of low-power embedded hardware.

Although the system operates successfully as a functional prototype, limitations remain in efficiency, wiring stability, and motor driver performance. These constraints identify clear pathways for future improvements, including PCB redesign, upgraded MOSFET-based motor drivers, integrated battery management, and expanded sensor-based feedback systems.

Overall, this project demonstrates a strong foundation in embedded systems engineering, combining theoretical electrical analysis with practical implementation and iterative debugging. It reflects the core engineering process of design, failure analysis, optimization, and system

refinement, resulting in a fully functional wireless control system that operates reliably under real-world conditions.

Appendix

The appendix contains supplementary technical material supporting the design, implementation, and validation of the wireless RC car embedded system. This section consolidates detailed references that are not part of the main analytical flow but are essential for full technical reproducibility and documentation completeness.

14.1 Full Circuit Schematics

The following schematics represent the complete hardware implementation of the system:

- ESP32 main controller schematic (motor control, LED, buzzer interface)
- L293D motor driver schematic (drive + steering subsystem integration)
- Power distribution schematic (5V logic rail and 4.5V motor rail separation)
- Peripheral control schematic (LED indicator and buzzer circuit)

These schematics define the complete electrical interconnection of all subsystems and reflect the final validated hardware configuration used during testing.

14.2 Pin Configuration Reference

A full pin mapping table is provided for reproducibility of the system wiring:

- ESP32 GPIO assignments for:
 - Motor drive direction control
 - PWM speed control signals
 - LED output control
 - Buzzer activation signal
- L293D input/output mapping:
 - IN1–IN4 directional control lines
 - ENA/ENB PWM enable channels
 - VCC1 logic supply (5V rail)
 - VCC2 motor supply (battery rail)

This mapping ensures consistent replication of system behavior across hardware builds.

14.3 Source Code

The system firmware is divided into two independent embedded programs:

ESP32 (Vehicle Controller)

- UDP packet receiver
- motor control logic (L293D interface)
- PWM speed handling
- LED and buzzer output control
- real-time command parsing

ESP32-S3 (Remote Controller)

- joystick and button input processing
- UDP packet generation
- Wi-Fi transmission system
- real-time control interface logic

Both programs operate asynchronously but remain synchronized through continuous wireless communication.

14.4 System Images and Prototype Documentation

The appendix includes visual documentation of the physical system:

- Breadboard-level prototype images
- Wiring layout photographs
- Assembled vehicle structure
- Control system integration overview

These images validate the real-world implementation of the schematics and confirm hardware consistency.

14.5 Communication Protocol Reference

The system uses UDP-based Wi-Fi communication with the following characteristics:

- Low-latency packet transmission

- No handshake overhead
- Continuous streaming of control data
- Packet structure includes:
 - direction values
 - speed values
 - peripheral state flags (LED, buzzer, stop command)

This communication method prioritizes responsiveness over guaranteed delivery, which is appropriate for real-time control systems.

14.6 Project Repository Reference

All project files, including schematics, firmware, and documentation, are stored in a structured repository for version control and future development.

- Source code (ESP32 + ESP32-S3)
- PDF engineering report
- schematic exports
- system diagrams
- testing media files

Acknowledgements

The development of this project required the integration of knowledge from multiple engineering disciplines including embedded systems programming, electronics, wireless communication, and circuit analysis. Throughout the design and debugging process, various software tools, technical resources, and engineering references contributed to the successful completion of the system.

Special acknowledgement is given to the open-source embedded development ecosystem surrounding the ESP32 platform, including community documentation, development libraries, and wireless networking resources that supported implementation and troubleshooting during development.

The project was designed and programmed using:

- Arduino IDE
- ESP32 development libraries
- UDP and Wi-Fi communication frameworks
- KiCad schematic design tools
- GitHub for project organization and documentation management

Additional acknowledgement is given to the iterative engineering process itself, which involved repeated testing, redesign, debugging, and optimization. The practical challenges encountered during development including power instability, communication issues, and system integration constraints provided valuable insight into real-world embedded systems engineering and significantly contributed to the overall learning experience.

This project represents both a technical implementation and a personal engineering milestone, combining theoretical electrical engineering concepts with hands-on system development and problem-solving.

Project Metadata

Project Title

ESP32-Based Wireless Embedded Control System for Dual-Motor Robotic Vehicle

Project Category

Wireless Embedded Systems Engineering Project

Project Type

Real-Time Wi-Fi Controlled Robotic Vehicle

Primary Technologies Used

- ESP32 Microcontroller
- ESP32-S3 Display Module
- UDP Wireless Communication
- PWM Motor Control
- Embedded C++ Programming
- L293D Motor Driver
- Breadboard Prototyping Architecture

Development Environment

- Arduino IDE
- ESP32 Board Support Package
- KiCad Schematic Design Software
- GitHub Documentation Environment

Communication Protocol

- Wi-Fi Access Point Architecture & UDP Packet-Based Communication

System Voltage Configuration

Subsystem	Voltage Source
ESP32 Logic Rail	5V Regulated Supply
L293D Logic Supply (VCC1)	5V
Motor Power Supply (VCC2)	4.5V (3× AA Batteries)
ESP32-S3 Remote Controller	USB-C Power Input

Motor Configuration

Motor Type	Function
Rear DC Motor	Forward / Reverse Drive
Front DC Motor	Steering Control

Firmware Architecture

Device	Primary Function
ESP32	Vehicle Control Unit
ESP32-S3	Wireless Remote Controller

Key Embedded Features

- Real-time wireless control
- PWM-based speed modulation
- Dual-motor directional control
- LED and buzzer peripheral management
- Power-domain separation
- Capacitor-assisted voltage stabilization

Prototype Revision

Version 1.0 — Breadboard Prototype Implementation

Planned Future Revision

Version 2.0 — PCB-Based Integrated Embedded Platform

Documentation Revision

Engineering Report Revision 1.0

Project Completion Date

May 2026

Repository and Documentation Status

- Schematics completed
- Source code completed
- Hardware block diagrams completed
- Software architecture completed
- Engineering analysis completed
- Testing documentation completed
- Final engineering report completed

Final Project Status

The project successfully achieved its primary objective of implementing a stable wireless embedded vehicle control platform using dual ESP32 microcontrollers communicating through low-latency UDP networking. The final prototype demonstrated functional real-time motor control, wireless communication stability, PWM speed regulation, and integrated peripheral operation within a fully documented engineering framework.